Emergency Vehicle Traffic Signal Preemption (EVTSP)

Integration Protocol: Mathematical Model, State Restoral & 3D Unity Architecture

1. Operational Principle & Integration with Corridor Model

In our corridor model, advance alerts reduced effective traffic density along forward segments. This module extends that foundation into an active **Intersection Preemption Layer**. When an emergency vehicle enters the detection radius of an upcoming intersection:

• **Priority Signal Preemption:** Regardless of current cycle timing (Red, Yellow, or Green), approaching direction transitions immediately to an intense flashing Glow Emergency Green.

• **Cross-Traffic Conflict Prevention:** All competing approaches immediately switch to Solid Red, preventing collisions and clearing conflicting movements.

• **Siren & Audio Doppler Simulation:** 3D positional audio emits a high-priority siren that dynamically attenuates over spatial distance.

• **Exact State Memory & Resumption:** The intersection caches its active state, elapsed time, and standard phase timings. Once departure trigger clears, normal cycles resume without starvation.

2. State Transition Machine

Normal Cycle State: Standard N/S and E/W green/yellow/red timer-based loop.
Preemption Trigger: Ambulance enters trigger zone → Store snapshot: (activePhase, timeRemaining) → Set all cross directions to RED → Set approach direction to EMERGENCY_GLOW_GREEN.
Post-Clearance Exit: Ambulance clears departure trigger → Restore snapshot state → Normal cycles resume.

3. Mathematical Integration with 1D Corridor Mechanics

At signal-controlled boundary segment s_k, effective velocity v(s_k) is constrained by signal factor $\sigma \in \{0, 1\}$:
$v(s_k) = \sigma \cdot v_{max} \cdot (1 - \rho_{eff}(s_k))$
Under standard operation, $\sigma = 0$ during red cycles, causing shockwave delay. The EVTSP protocol forces $\sigma = 1$ upon entry into horizon H, eliminating queue-clearing stagnation.

4. Unity Component Breakdown

Unity Scene Architecture	Audio & Visual Assets
Intersection Controller: Manages 4-way intersection logic and caches phase history. **Signal Light Objects:** 3D traffic light heads with emissive materials and point lights. **Trigger Zones:** Convex Box/Sphere Colliders marked as IsTrigger = true.	**3D Audio Source:** Logarithmic Rolloff (min 5m, max 100m) attached to ambulance. **HDR Emissive Materials:** Materials utilizing _EmissionColor to produce real-time bloom glow in Unity URP.

TrafficSignalController.cs (C# Unity Core)

Multi-Way Signal Controller with State Memory & Dynamic Emergency Overrides

5. Implementation: TrafficSignalController.cs

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public enum SignalState { Red, Yellow, Green, EmergencyGlowGreen }
public enum ApproachDirection { North, South, East, West }

[System.Serializable]
public class SignalHead {
    public ApproachDirection direction;
    public MeshRenderer redBulb, yellowBulb, greenBulb;
    public Light emergencySpotlight;
    [HideInInspector] public SignalState currentState;
}

public class TrafficSignalController : MonoBehaviour {
    public List<SignalHead> signals;
    public float greenDuration = 10.0f, yellowDuration = 3.0f;
    public Color redColor = Color.red, yellowColor = Color.yellow, greenColor = Color.green;
    public Color emergencyGlowColor = new Color(0.1f, 1.0f, 0.4f) * 4.5f;

    private bool isEmergencyActive = false;
    private ApproachDirection savedGreenDirection;
    private float savedTimeInCycle = 0f;
    private bool savedIsYellow = false;
    private Coroutine activeCycleCoroutine;

    void Start() { activeCycleCoroutine = StartCoroutine(NormalCycleRoutine()); }

    private IEnumerator NormalCycleRoutine() {
        while (!isEmergencyActive) {
            // N/S Green
            savedGreenDirection = ApproachDirection.North; savedIsYellow = false;
            SetDirectionState(ApproachDirection.North, SignalState.Green);
            SetDirectionState(ApproachDirection.South, SignalState.Green);
            SetDirectionState(ApproachDirection.East, SignalState.Red);
            SetDirectionState(ApproachDirection.West, SignalState.Red);
            for (savedTimeInCycle = 0; savedTimeInCycle < greenDuration; savedTimeInCycle += Time.deltaTime) yield return null;

            // N/S Yellow
            savedIsYellow = true;
            SetDirectionState(ApproachDirection.North, SignalState.Yellow);
            SetDirectionState(ApproachDirection.South, SignalState.Yellow);
            for (savedTimeInCycle = 0; savedTimeInCycle < yellowDuration; savedTimeInCycle += Time.deltaTime) yield return null;

            // E/W Green
            savedGreenDirection = ApproachDirection.East; savedIsYellow = false;
            SetDirectionState(ApproachDirection.North, SignalState.Red);
            SetDirectionState(ApproachDirection.South, SignalState.Red);
            SetDirectionState(ApproachDirection.East, SignalState.Green);
            SetDirectionState(ApproachDirection.West, SignalState.Green);
            for (savedTimeInCycle = 0; savedTimeInCycle < greenDuration; savedTimeInCycle += Time.deltaTime) yield return null;

            // E/W Yellow
            savedIsYellow = true;
            SetDirectionState(ApproachDirection.East, SignalState.Yellow);
            SetDirectionState(ApproachDirection.West, SignalState.Yellow);
            for (savedTimeInCycle = 0; savedTimeInCycle < yellowDuration; savedTimeInCycle += Time.deltaTime) yield return null;
        }
    }

    public void TriggerEmergencyPreemption(ApproachDirection approachingFrom) {
        if (isEmergencyActive) return;
        isEmergencyActive = true;
        if (activeCycleCoroutine != null) StopCoroutine(activeCycleCoroutine);

        foreach (var sig in signals) ApplyState(sig, SignalState.Red);
        SignalHead target = signals.Find(s => s.direction == approachingFrom);
        if (target != null) ApplyState(target, SignalState.EmergencyGlowGreen);
    }

    public void ReleaseEmergencyPreemption() {
        if (!isEmergencyActive) return;
        isEmergencyActive = false;
        foreach (var sig in signals) ApplyState(sig, SignalState.Red);
        activeCycleCoroutine = StartCoroutine(ResumeCycleRoutine());
    }

    private IEnumerator ResumeCycleRoutine() {
        float remaining = (savedIsYellow ? yellowDuration : greenDuration) - savedTimeInCycle;
        if (remaining > 0.5f) {
            SignalState resumeState = savedIsYellow ? SignalState.Yellow : SignalState.Green;
            ApproachDirection dir = savedGreenDirection;
            SetDirectionState(dir, resumeState);
            SetDirectionState(dir == ApproachDirection.North ? ApproachDirection.South : ApproachDirection.West, resumeState);
            yield return new WaitForSeconds(remaining);
        }
        activeCycleCoroutine = StartCoroutine(NormalCycleRoutine());
    }
}
```

AmbulanceVehicle.cs & IntersectionTrigger.cs

Siren Audio Synthesis, Rigidbody Movement & Collision Triggers

6. Implementation: AmbulanceVehicle.cs

```
using UnityEngine;

[RequireComponent(typeof(AudioSource))]
public class AmbulanceVehicle : MonoBehaviour {
    public float targetSpeed = 15.0f; // m/s
    public bool isRespondingToEmergency = true;
    public AudioClip sirenClip;
    [Range(0f, 1f)] public float sirenVolume = 1.0f;
    public float minAudioDistance = 5.0f, maxAudioDistance = 120.0f;

    private AudioSource audioSource;
    private Rigidbody rb;

    void Awake() {
        rb = GetComponent<Rigidbody>();
        audioSource = GetComponent<AudioSource>();
        audioSource.clip = sirenClip;
        audioSource.loop = true;
        audioSource.volume = sirenVolume;
        audioSource.spatialBlend = 1.0f; // 3D sound
        audioSource.dopplerLevel = 1.5f;
        audioSource.rolloffMode = AudioRolloffMode.Logarithmic;
        audioSource.minDistance = minAudioDistance;
        audioSource.maxDistance = maxAudioDistance;
    }

    void Start() {
        if (isRespondingToEmergency && sirenClip != null) audioSource.Play();
    }

    void FixedUpdate() {
        if (isRespondingToEmergency) {
            Vector3 targetVelocity = transform.forward * targetSpeed;
            if (rb != null) rb.linearVelocity = new Vector3(targetVelocity.x, rb.linearVelocity.y, targetVelocity.z);
            else transform.position += targetVelocity * Time.fixedDeltaTime;
        }
    }
}
```

7. Implementation: IntersectionTrigger.cs

```
using UnityEngine;

public class IntersectionTrigger : MonoBehaviour {
    public TrafficSignalController controller;
    public ApproachDirection entryDirection;

    private void OnTriggerEnter(Collider other) {
        AmbulanceVehicle ambulance = other.GetComponentInParent<AmbulanceVehicle>();
        if (ambulance != null && ambulance.isRespondingToEmergency) {
            controller.TriggerEmergencyPreemption(entryDirection);
        }
    }

    private void OnTriggerExit(Collider other) {
        AmbulanceVehicle ambulance = other.GetComponentInParent<AmbulanceVehicle>();
        if (ambulance != null && ambulance.isRespondingToEmergency) {
            controller.ReleaseEmergencyPreemption();
        }
    }
}
```

Interactive Virtual 4-Way Simulation (HTML5/Web Audio)

Real-Time Canvas Simulation with Synthesized Doppler Siren & Signal Preemption

8. Complete Simulation Source: intersection_simulation.html

The standalone HTML5 simulator replicates the complete Unity preemption mechanics, featuring a synthesized Web Audio emergency siren, dynamic 4-way signals, glowing priority indicators, and automated state resumption. The file is accessible locally at **intersection_simulation.html**.

```
// Web Audio Doppler Synthesizer & Dispatch Logic
function initAudio() {
  if (audioCtx) return;
  audioCtx = new (window.AudioContext || window.webkitAudioContext)();
  sirenOsc = audioCtx.createOscillator();
  sirenGain = audioCtx.createGain();
  sirenOsc.type = 'sawtooth';
  sirenOsc.frequency.setValueAtTime(650, audioCtx.currentTime);
  sirenGain.gain.setValueAtTime(0, audioCtx.currentTime);
  sirenOsc.connect(sirenGain);
  sirenGain.connect(audioCtx.destination);
  sirenOsc.start();
}

function dispatchAmbulance(direction) {
  initAudio();
  approaching = direction;
  amb.active = true;
  amb.dir = direction;
  if (direction === 'West') { amb.x = 20; amb.y = 265; amb.vx = 2.6; amb.vy = 0; }
  if (direction === 'South') { amb.x = 235; amb.y = 480; amb.vx = 0; amb.vy = -2.6; }
}

function updatePreemptionState(dist) {
  if (dist < 180 && !preemption && dist > 20) {
    preemption = true;
    savedPhase = phase; savedTimer = phaseTimer;
    statusPanel.innerText = 'PREEMPTION ACTIVE: ' + amb.dir + ' Granted Emergency Green!';
  }
}
```

Unity Setup Runbook & Technical Defense Guide

Scene Configuration, Universal Render Pipeline (URP) Bloom & Viva Talking Points

9. Step-by-Step Unity Setup Runbook

1. **Scene & Corridor Hierarchy:** Create 4 intersecting road planes. Attach TrafficSignalController.cs to an empty GameObject named Intersection_Controller.

2. **Signal Heads & Meshes:** Create 4 signal post prefabs with Red, Yellow, and Green bulb spheres. Assign materials supporting Emission.

3. **Bloom Glow Configuration:** In Unity URP, add Global Volume → Bloom (Intensity = 1.5, Threshold = 0.9). HDR emission multipliers generate realistic optical halos.

4. **Emergency Triggers:** Place trigger colliders 50–80m before the intersection with IsTrigger = true and attach IntersectionTrigger.cs.

5. **Ambulance Prefab:** Attach AmbulanceVehicle.cs, Rigidbody, AudioSource, and set Spatial Blend = 1.0 (3D) for Doppler attenuation.

10. Viva Defense & Technical Q&A;

Q: How does the system prevent cross-traffic accidents? **Answer:** The instant emergency detection fires, all competing signal phases are interrupted and immediately forced to Solid Red. An optional 1.5s all-red clearance interval clears vehicles currently inside.	**Q: How does cycle memory prevent post-clearance starvation?** **Answer:** The controller caches active phase and elapsed time. Once departure trigger clears, the system smoothly resumes the remaining phase duration rather than resetting to Phase 0.

11. Final Integration Checklist

• [x] TrafficSignalController.cs integrated with HDR emission control and state memory.

• [x] AmbulanceVehicle.cs configured with 3D audio siren and forward kinematics.

• [x] IntersectionTrigger.cs detecting arrival and departure boundaries.

• [x] Interactive browser simulator (intersection_simulation.html) with synthesized Web Audio siren and visual bloom.